

1. 哪些函数不能在中断中使用？

答：动态内存分配函数：例如 `malloc()`、`free()` 等函数，因为在中断中动态分配和释放内存可能导致内存碎片问题，甚至造成内存泄漏。

阻塞函数：包括像 `sleep()`、`delay()` 等需要等待一段时间的函数，因为在中断服务程序中阻塞会影响系统的实时性。

信号量操作函数：如 `sem_wait()`、`sem_post()` 等，因为这些函数可能会导致死锁或竞争条件。

文件操作函数：例如 `fopen()`、`fclose()` 等，因为在中断服务程序中访问文件可能会引起不可预测的结果。

2. 什么叫不可重入函数？

答：不可重入函数是指在同一时间只能被一个任务或线程执行的函数，不可重入函数会使用一些全局变量或静态变量来保存状态信息，如果在该函数执行过程中被中断，并在中断处理程序或其他任务中再次调用该函数，那么全局状态可能会被破坏，导致函数无法正确执行。

3. 如果在一个中断里面有一个变量，其他线程里面需要读取这个变量要注意什么？

答：我会用 `volatile` 对这个变量进行修饰，在使用这个变量的其他线程中创建一个互斥量，获得锁读取这个变量的，然后释放锁。

```
volatile int sharedVariable = 0;
```

中断

```
{
    xSemaphoreTakeFromISR(xSemaphore, NULL);
    sharedVariable = 42;
    xSemaphoreGiveFromISR(xSemaphore, NULL);
}
```

4. 定时器 PWM？

答：周期由定时器的计数值决定。占空比表示高电平状态（信号为 1）在一个周期内所占的时间比例，它决定了 PWM 信号的电平平均值。当定时器的计数值小于占空比乘以周期时，PWM 信号处于高电平状态。当计数值大于占空比乘以周期时，PWM 信号处于低电平状态。

5. 定时器有几种用法

答：1.最常用的就是延迟 `delay`

2.定时中断，`tick` 中断

3.PWM 输出控制电机的

4.计时计数

6. `const` 关键字？

答：1.将变量修饰成常量，放在 `flash` 上

2.修饰函数参数，防止其他函数调用这个函数的时候，把里面的值修改了。

3.修饰指针 `const int *p`、`int *const p`

7. static 关键字?

答: 1.static 修饰的变量放在静态存储区, 只初始化一次, 一般是写状态机的时候, 一个任务时间消耗的时间比较大, 就可以分段进行
2. static 修饰的函数, 让这个函数只在这个文件里面生效, 其他文件要使用这个函数只能使用我暴露出去的结构体, 或者使用 **extern** 去调用, 更加安全

8. volatile 关键字?

答: 1.使用 **volatile** 一般就是使用物理内存的时候去告诉 **cpu** 说不可以去优化他, 这样确保每次都是最新的值
2.调试的时候, 有些操作需要记录下来, 使用 **volatile** 修饰, 可以不会被 **cpu** 优化了
3.常见的就是不同线程中的通信, 这样可以保证变量的准确性
4.中断里面用的变量一般都建议用 **volatile** 去修饰, 防止被其他的中断改了。

9. 数组和链表的区别?

答: 1.数组要在块连续的空间上, 链表由节点组成。
2.链表的内存开销比较大
3.数组可以快速访问任意位置的元素, 链表只能按顺序访问

10. 通用寄存器包括 R0~R15, 可以分为具体哪三类?

答: R0-R3(传入函数参数, 传出函数返回值), R4-R12(存局部变量), R13(SP 栈指针), R14(LR 寄存器), R15(PC 寄存器)

11. 如何对绝对地址 0x100000 赋值?

答: c 语言

```
volatile uint32_t* address = (volatile uint32_t*)0x100000;  
*address = value;
```

汇编

```
LDR R0, =0x100000    ; 将地址 0x100000 加载到寄存器 R0  
LDR R1, =value       ; 将 value 加载到寄存器 R1  
STR R1, [R0]         ; 将 R1 的值存储到 R0 指向的地址
```

12. stm32 寄存器详解

答: R0-R3: 传入函数参数, 传出函数返回值; R4-R12: 存放函数的局部变量; R13: SP 栈指针; R14(LR): 保存返回地址; R15(PC): 用于存储下条指令的地址

13. 什么是交叉编译?

答: 交叉编译是指使用一台计算机来编译另一种不同架构的目标平台上的程序。

